

Mathematical Reasoning Ability of Large Language Models: Integrated Project Report

Group: Lakers

Members:

- 1. LiXianlin 60003835**
- 2. ZhangKaige 60065468**
- 3. WangBoheng 59624730**
- 4. Ho Ka Chun 55216809**
- 5. HaoJiacheng59931279**
- 6. Huangdehao 60021619**
- 7. XieYi 59658209**

Step 1. Data Collection and Cleaning

We built a preprocessing pipeline that integrates several well-known mathematics benchmark datasets into a single, consistent JSONL format for evaluation. The pipeline aggregates the GSM8K test set, the full Hendrycks MATH test set, the MATH-500 subset, and the AIME 2024 problems. Where possible, answers are normalized to a common representation, and each run outputs timestamped files under the /trained dataset directory to avoid overwriting previous results. Due to the anti-bot protections enforced by Art of Problem Solving (AoPS), the AIME problems are supplied manually through a local CSV file (aime2024_input.csv). If this CSV is not available, the pipeline falls back to an official answer table embedded in the script so that processing can still proceed without attempting to scrape protected pages. The project provides both a command-line script (preprocess.py) and an interactive Jupyter notebook (preprocess.ipynb), which implement the same logic and allow users to run the pipeline either end-to-end or step by step for closer inspection.

The preprocessing code is organized around small, clearly documented helper functions. These include `norm_number`, which standardizes numeric strings by stripping whitespace and removing thousands separators; `extract_after_hashes`, which retrieves final numeric answers from GSM8K solutions

that use the ##### delimiter; and `extract_boxed`, which extracts LaTeX boxed answers and otherwise searches for a clearly marked answer line in the solution text. Together, these utilities make the pipeline resilient to common formatting differences across datasets, such as extra spaces, commas in numbers, or LaTeX-style answer notation.

Output handling is designed to be safe and repeatable. The `write_jsonl` function always writes to a timestamped JSONL file so that repeated runs do not clobber existing outputs. The `get_math_tags` helper assigns consistent subject and difficulty tags to MATH problems, while `install_dependencies` attempts to automatically install any missing Python dependencies such as `datasets` or `accelerate`. This simplifies setup for first-time users, although running inside a virtual environment is recommended to avoid modifying the global Python installation.

Each dataset is processed through its own dedicated function. The `process_gsm8k` function loads the `openai/gsm8k` test split from Hugging Face, extracts final answers using `extract_after_hashes`, and outputs standardized records containing fields such as `problem_id`, normalized answer, reference URL, and metadata describing the answer format. The `process_math` function iterates through the subject splits of the Hendrycks MATH dataset, recovers final answers from solution text, and applies subject- and level-based tags. The `process_math500` function performs equivalent normalization for the official MATH-500 test split.

The `process_aime2024` function is specifically designed to accommodate AoPS access restrictions. It first checks for the presence of `aime2024_input.csv` and reads problems, along with optional answers, from this file if available. The CSV parsing logic is intentionally defensive: it prefers header-based parsing, but if rows have inconsistent lengths, it reconstructs fields by treating the first column as the index, the last columns as answer, `has_diagrams`, and URL, and joining any intermediate columns into the problem statement. When an answer cell is empty or contains non-numeric text, the function falls back to the built-in official answer list to ensure completeness. Each AIME entry stores its normalized answer as a three-digit string (000-999) and includes metadata describing the fixed-format answer type, with a default AoPS reference URL when none is supplied.

To support manual data entry, the helper function `create_aime_template` generates a starter CSV file (`aime2024_input.csv`) with placeholder problem text and empty answer fields. Overall execution is exposed in two parallel forms: the `main()` entry point in `preprocess.py`, which installs dependencies, prepares the output directory, runs all dataset processors, and prints a summary; and the `preprocess.ipynb` notebook, which mirrors the same steps for interactive experimentation.

Overall, the project provides a robust and repeatable preprocessing workflow that aligns multiple public mathematics datasets under a unified JSONL schema. It combines defensive parsing and normalization to handle messy source formats, preserves reproducibility through timestamped outputs, and respects anti-scraping constraints by relying on a manual CSV import for AIME 2024 rather than automated data collection.

Step 2. Prompt Design and Unification

We implemented a prompting and reasoning module based on the datasets produced by the preprocessing pipeline. The prompting system is implemented in `prompt.py` and is organized around reusable prompt templates and a unified execution interface. Each prompt method uses a formatted instruction that includes the problem text while enforcing a standardized answer format. This constraint simplifies future evaluation because model outputs can be automatically parsed to extract the final answer. All prompting strategies share a common system instruction that encourages careful reasoning and ensures consistent behavior across different experimental conditions.

Five prompting strategies are supported in the current implementation. The Direct Prompt serves as a baseline where the model is asked to solve the problem without explicit reasoning instructions. Chain-of-Thought (CoT) prompting encourages the model to produce step-by-step reasoning before generating the final answer, which can improve performance on multi-step problems. Least-to-Most prompting decomposes complex tasks into smaller subproblems and solves them sequentially, encouraging the model to build a reasoning process. Self-Consistency improves robustness by sampling multiple CoT reasoning paths and selecting the most frequent answer through majority voting. Finally, Plan-and-Solve prompting introduces an explicit planning stage.

All prompting methods are accessed through a unified interface, `run_prompt_method`, which takes the model, tokenizer, question text, and the selected prompting strategy as inputs. This abstraction allows the experiment runner to switch between prompting methods while keeping the dataset and model configuration fixed. The design ensures that the entire project remains reproducible, extensible, and easy to analyze.

Step 3. Experiments with Qwen

This part presents a systematic empirical evaluation of the mathematical reasoning performance of Qwen2.5-Math-1.5B, a lightweight open-source large language model specialized for mathematical tasks. The experiment was conducted on two standard mathematical reasoning benchmarks: the GSM8K test set, which contains elementary arithmetic word problems, and the MATH-500 dataset, which contains

challenging competition-style math problems. Three widely adopted prompting strategies were compared: Chain-of-Thought (CoT), Least-to-Most (LtM), and Self-Consistency (SC, $k=5$).

Two core metrics were measured for all experiments. The first was accuracy, defined as the proportion of questions answered correctly and used to reflect reasoning correctness. The second was average response length, defined as the mean character count of model outputs and used to reflect the verbosity and detail of reasoning processes.

The experiments used the following setup. The model was Qwen2.5-Math-1.5B, with a parameter size of 1.5 billion. It is a causal language model fine-tuned explicitly for mathematical reasoning tasks. The model ran in FP16 precision on a single NVIDIA GPU with CUDA acceleration, optimized with `torch.compile()` for inference speedup. The datasets included 500 GSM8K test samples and 500 MATH-500 samples. In terms of difficulty, GSM8K represents low-difficulty K-12 grade school arithmetic word problems, while MATH-500 represents high-difficulty competition-style mathematics, including algebra, number theory, calculus, and combinatorics.

The result summary is as follows:

Dataset	Prompt Method	Total Samples	Correct	Accuracy	Avg Response Length
GSM8K	CoT	500	274	54.8%	1939.51
GSM8K	Least-to-Most	500	229	45.8%	2234.96
GSM8K	Self-Consistency ($k=5$)	500	309	61.8%	10227.44
MATH-500	CoT	500	130	26.0%	1910.05
MATH-500	Least-to-Most	500	105	21.0%	2114.98
MATH-500	Self-Consistency ($k=5$)	500	138	27.6%	10152.52

Several key findings emerge from these results. First, Self-Consistency consistently achieves the highest accuracy across both datasets, outperforming CoT and LtM by a clear margin. On GSM8K, SC reaches 61.8% accuracy, while CoT and LtM achieve 54.8% and 45.8%, respectively. On MATH-500, SC scores 27.6%, compared with 26.0% for CoT and 21.0% for LtM. Second, the accuracy gains of SC

come at the cost of significantly longer outputs, with average lengths exceeding 10,000 characters on both datasets, which is about five times longer than CoT and LtM. Third, the model shows a substantial performance gap between the two benchmarks: accuracy on GSM8K is more than double the accuracy on MATH-500 for all prompting methods, highlighting the challenge of advanced mathematical reasoning for small LLMs.

The discussion of the Qwen experiments leads to several conclusions. Self-Consistency provides the highest accuracy but also incurs much higher inference cost, which makes it suitable when correctness is prioritized over speed. CoT offers a better efficiency-accuracy trade-off for resource-constrained deployment. Least-to-Most underperforms CoT in this small model, suggesting that the benefits of decomposition may depend on a stronger inherent ability to break down problems. The results also show that longer responses do not necessarily guarantee better reasoning quality. LtM produces slightly longer outputs than CoT but still performs worse on both datasets.

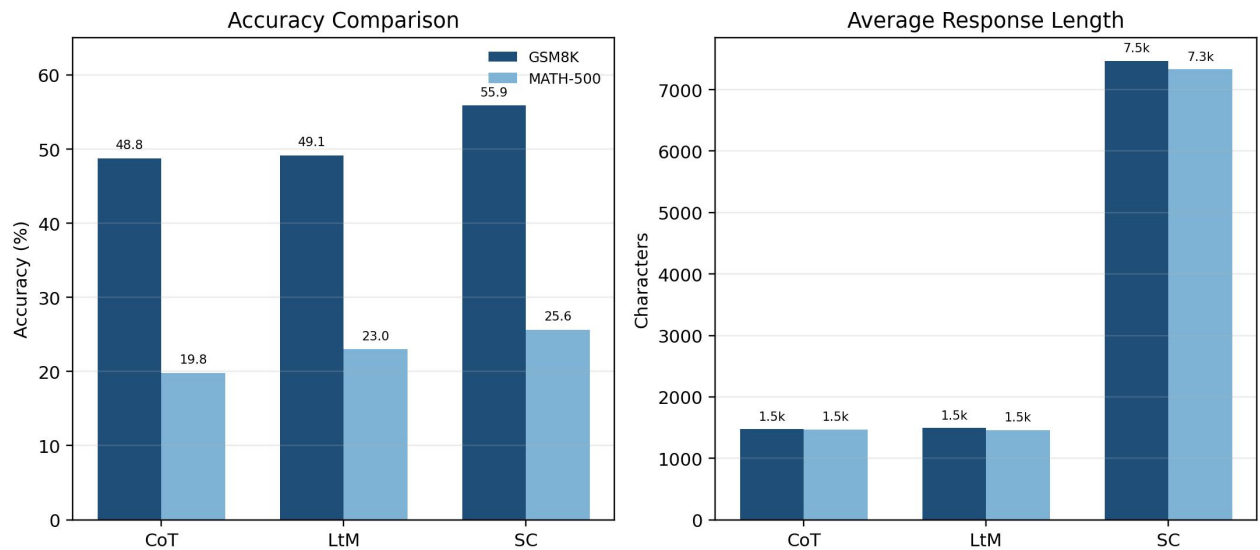
Step 4. Experiments with DeepSeek

This section documents the DeepSeek main experiments on MATH-500 and GSM8K. The implemented pipeline is built around the DeepSeek-R1-Distill-Qwen-1.5B model and the two datasets that belong to the main experiment package. Each item is loaded from the unified JSONL datasets prepared by the preprocessing pipeline. For every question, the experiment script records the dataset name, question identifier, prompt method, raw model output, extracted final answer, correctness flag, response length, and generation time. This supports later re-evaluation without rerunning the model.

The experimental configuration is summarized as follows. The model was DeepSeek-R1-Distill-Qwen-1.5B. The datasets in this sub-report were MATH-500, with 500 questions, and GSM8K, with 800 final evaluated questions. The prompting methods were CoT, Least-to-Most, and Self-Consistency with $k = 5$. The core required metrics were accuracy and response length, while runtime was additionally reported as a practical cost signal. Automatic evaluation first extracts a candidate final answer from the raw output. The extractor prioritizes explicit answer markers such as boxed answers or answer phrases, then applies conservative cleanup and dataset-specific matching. For GSM8K, numeric correctness is the main criterion. For MATH-500, the matcher tries normalized text matching, numeric matching, and symbolic-equivalence checks where possible.

The final experiment run covered all 500 MATH-500 questions and 800 GSM8K questions contained in the final result files. Each question was tested under CoT, Least-to-Most, and Self-Consistency. The final accuracy table is shown below:

Dataset	CoT	LtM	SC	Best Method	Observation
GSM8K	48.75%	49.13%	55.87%	SC	Best overall accuracy
MATH-500	19.80%	23.00%	25.60%	SC	Harder symbolic benchmark



The final results show a clear ranking pattern. On GSM8K, Self-Consistency achieves the best accuracy at 55.87%, followed by Least-to-Most at 49.13% and CoT at 48.75%. On MATH-500, Self-Consistency again performs best at 25.60%, followed by Least-to-Most at 23.00% and CoT at 19.80%. Thus, the same method wins on both datasets, although the size of the improvement differs. GSM8K remains substantially easier than MATH-500 for this 1.5B model, suggesting that symbolic reasoning and answer formatting remain challenging on the harder benchmark.

The project brief requires response length as a core metric, and the final logs also allow generation time analysis. Both cost indicators point to the same conclusion: the most accurate method is also the most expensive. Self-Consistency increases average response length from about 1.47k-1.49k characters to roughly 7.33k-7.47k characters, which is about five times longer than CoT and Least-to-Most. The runtime increase is similarly large.

Dataset	Metric	CoT	LtM	SC
GSM8K	Avg response length	1472.40	1491.45	7468.57
GSM8K	Avg generation time (s)	17.91	18.10	90.44
MATH-500	Avg response length	1468.95	1458.19	7329.16
MATH-500	Avg generation time (s)	20.26	19.84	102.10

Therefore, the choice of prompt method depends on the goal of the application. If the main objective is the highest possible accuracy, Self-Consistency is the strongest setting in this report. If inference cost matters more, CoT or Least-to-Most may be more practical, because their accuracy gap is much smaller than their cost gap.

An important finding from the DeepSeek experiments is that not all errors come from completely incorrect reasoning. Many MATH-500 failures occur because the model does not end with a clean and stable final answer, even when the middle of the reasoning chain looks partially relevant. Some outputs end with an incomplete symbolic phrase, some remain in narrative reasoning mode instead of switching to a final answer statement, and some are truncated or malformed so that the extracted result becomes only a fragment. These issues affect MATH-500 more strongly than GSM8K because MATH-500 contains more symbolic answers and is less forgiving when the final format is unstable.

Step 5. Experiments on AIME with Both Models

The AIME 2024 subset consists of 15 challenging competition problems. The goal of this stage was to evaluate two small-scale mathematical reasoning models, Qwen2.5-Math-1.5B and DeepSeek-R1-Distill-Qwen-1.5B, using three prompting strategies: Chain-of-Thought (CoT), Least-to-Most (LtM), and Self-Consistency (SC, $k=5$). The outputs generated in this stage serve as the foundation for later accuracy computation, response length analysis, and error classification.

The input dataset was provided in a unified JSONL format produced by the preprocessing stage. The file was `unified_aime2024_i_official_20260311_191357.jsonl`. Each entry contains `problem_id`, `question`, `answer_normalized`, which is the official three-digit integer answer, and `metadata`. All 15 AIME 2024

problems were included, with problem texts manually imported from a local CSV as described in the preprocessing pipeline.

Both models were loaded in half precision (`torch.float16`) and automatically placed on the available GPU using `device_map="auto"`. The generation parameters were `max_new_tokens = 512`, `temperature = 0.7`, `top_p = 0.9`, and `do_sample = True`, which is necessary for self-consistency. The prompting module from Step 2, `prompt.py`, was reused through the unified function `run_prompt_method(model, tokenizer, question, method)`. This function returns the prompting strategy, the raw generated output, and the extracted numeric answer, or `None` when extraction fails. Answer extraction relies on a regular expression that looks for Answer: `<number>` at the end of the output.

Experiments were run on Google Colab with a Tesla T4 GPU with 16 GB VRAM to ensure sufficient speed and stability. The whole process took approximately 1.5 hours. A custom script, `run_aime_experiments.py`, orchestrated the experiment. It loaded the dataset from the provided JSONL file; for each model, it loaded the tokenizer and model, iterated over all 15 problems, and for each problem called `run_prompt_method` for CoT, LtM, and SC. It recorded the result, including the generated output, extracted answer, ground truth, generation time, and status (success, no_answer_extracted, or error), and wrote each record immediately to a JSONL file, one file per model, to avoid data loss. For self-consistency, the five sampled outputs were concatenated with separators for readability.

Each result file contains one JSON object per (model, problem, method) combination. The recorded fields include `model`, `problem_id`, `question`, `ground_truth`, `method`, `final_answer`, `raw_output`, `generation_time_sec`, `status`, and `error_msg` when an error occurs. The two deliverables uploaded to the team repository under `experiments/results/` were `Qwen2.5-Math-1.5B_aime_results.jsonl` and `DeepSeek-R1-Distill-Qwen-1.5B_aime_results.jsonl`.

Several observations were noted during execution. Both models occasionally produced outputs that did not follow the exact Answer: `<number>` format, leading to extraction failures and the status `no_answer_extracted`. These cases were flagged for manual review, and the later evaluation stage used a more flexible extraction strategy to recover intended numeric answers where possible. In self-consistency, the majority vote sometimes produced no answer when fewer than three answers matched. AIME problems also triggered long reasoning chains, but the 512-token limit was sufficient for all runs. No runtime errors occurred during execution; all 15 problems x 3 methods x 2 models completed successfully.

Step 6. AIME Evaluation

The final AIME evaluation compares the outputs of both models across all prompting methods. In total, each model produced 45 results, corresponding to 15 problems evaluated with CoT, LtM, and SC. The summary metrics are shown below:

Model	Count	Correct	Accuracy	Chars Mean	Chars Median	Words Mean	Words Median
DeepSeek-R1-Distill-Qwen-1.5B	45	0	0.0000	5985.2	2809	1001.0	460
Qwen2.5-Math-1.5B	45	1	0.0222	5140.2	2521	772.5	427

The overall accuracy on AIME was very low for both models. DeepSeek-R1-Distill-Qwen-1.5B achieved 0/45, while Qwen2.5-Math-1.5B achieved 1/45, corresponding to an accuracy of 0.0222. The results indicate that the AIME 2024 problems are extremely challenging for both 1.5B-scale models, even when multiple prompting strategies are used.

Error category analysis further clarifies the main failure modes:

Model	Error Category	Count
DeepSeek-R1-Distill-Qwen-1.5B	Format/noise issue	36
DeepSeek-R1-Distill-Qwen-1.5B	No extractable answer	8
DeepSeek-R1-Distill-Qwen-1.5B	Reasoning/calculation error	1
Qwen2.5-Math-1.5B	Format/noise issue	25
Qwen2.5-Math-1.5B	Reasoning/calculation error	15
Qwen2.5-Math-1.5B	Template residue	3
Qwen2.5-Math-1.5B	No extractable answer	1

For DeepSeek, format and noise issues dominated the errors, accounting for 36 of 45 cases. For Qwen, the error profile was more mixed, with both format/noise issues and reasoning or calculation errors contributing substantially. This suggests that on AIME, failure is driven not only by difficult mathematical reasoning but also by instability in answer formatting and output cleanup.

The extraction source distribution also reflects this pattern:

Model	Extraction Source	Count
DeepSeek-R1-Distill-Qwen-1.5B	tail_last_number	36
DeepSeek-R1-Distill-Qwen-1.5B	none	8
DeepSeek-R1-Distill-Qwen-1.5B	boxed	1
Qwen2.5-Math-1.5B	tail_last_number	25
Qwen2.5-Math-1.5B	the_answer	7
Qwen2.5-Math-1.5B	boxed	6
Qwen2.5-Math-1.5B	final_answer	3
Qwen2.5-Math-1.5B	answer_line	3
Qwen2.5-Math-1.5B	none	1

Accuracy by method shows that only Qwen with Self-Consistency achieved one correct answer:

Model	Method	Correct/Total	Accuracy
DeepSeek-R1-Distill-Qwen-1.5B	cot	0/15	0.0000
DeepSeek-R1-Distill-Qwen-1.5B	ltm	0/15	0.0000
DeepSeek-R1-Distill-Qwen-1.5B	sc	0/15	0.0000
Qwen2.5-Math-1.5B	cot	0/15	0.0000
Qwen2.5-Math-1.5B	ltm	0/15	0.0000
Qwen2.5-Math-1.5B	sc	1/15	0.0667

Representative wrong cases reveal the types of errors made by the two models. For example, DeepSeek on aime_2024_i_01 with CoT predicted 4 for a ground truth of 204, which was classified as a format/noise issue. On aime_2024_i_02 with SC, DeepSeek produced no extractable answer for a ground truth of 25. On aime_2024_i_11 with LtM, it predicted 92 instead of 371, representing a reasoning or

calculation error. Qwen showed similar issues: on aime_2024_i_01 with CoT, it predicted 2 instead of 204; on the same problem with SC, it predicted -1; on aime_2024_i_03 with SC, it predicted 404 instead of 809, which was classified as template residue; and on aime_2024_i_10 with CoT, it produced no extractable answer for a ground truth of 113.

Overall, the AIME evaluation shows that both models struggle severely on this benchmark. Compared with GSM8K and MATH-500, AIME places much stronger demands on exact mathematical reasoning, stable output formatting, and reliable extraction of final answers. The results suggest that improving final-answer control and reducing formatting noise may be just as important as improving reasoning depth when evaluating small mathematical LLMs on competition-level tasks.